Neural Networks, Representation Learning, CNNs, Sequence Models, and Transformers

Gabriel Wendell Celestino Rocha

How to use these slides

Structure of Wednesday

- **Slot 1:** Neural networks and representation learning.
- **Slot 2:** CNNs, sequence models, and transformers.

Goal of the day

- Understand what neural networks compute.
- See why deep learning is representation learning.
- Match architectures to astrophysical data geometry.

Guiding question

What structure should the model be allowed to learn?

Suggested timing

Slot	Content	Time
1A	From linear models to neural networks	10 min
1B	Forward pass, loss, backpropagation, optimization	20 min
1C	Regularization, normalization, and representation learning	20 min
1D	Micro-discussion: what did the network learn?	5-10 min
2A	Convolutional neural networks and equivariance	20 min
2B	Sequence models for light curves and spectra	15 min
2C	Attention and transformers	20 min
2D	Architecture choice and scientific reliability	5-10 min

The goal is not to memorize architectures, but to understand their assumptions.

Learning outcomes for Wednesday

By the end of this lesson, you should be able to:

1. Write the forward pass of a multilayer neural network.
2. Explain why nonlinear activations are necessary.
3. Interpret backpropagation as repeated use of the chain rule.
4. Recognize overfitting in neural networks and name common controls.
5. Explain representation learning using embeddings and latent spaces.
6. State what inductive bias a CNN introduces.
7. Distinguish 1D CNNs, RNNs/GRUs/LSTMs, TCNs, and transformers.
8. Choose a plausible architecture for images, spectra, light curves, and catalogs.
9. Explain why good test performance is not automatically physical understanding.

From Monday to Wednesday

Monday: statistical learning

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}[f_\theta(\boldsymbol{x}_i), y_i] \ .$$

- data, labels, loss;
- validation and generalization;
- metrics and leakage.

Wednesday: representation learning

$$\boldsymbol{x} \longmapsto \boldsymbol{h}_1 \longmapsto \boldsymbol{h}_2 \longmapsto \cdots \longmapsto \hat{y} \ .$$

- each layer transforms the data;
- useful intermediate variables are learned;
- architecture encodes assumptions.

Why go beyond linear models?

A linear model predicts

$$\hat{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b} \ .$$

It can be powerful if the features already make the problem simple.

But astrophysical data often contain:

- nonlinear relations;
- interactions between observables;
- hierarchical structure;
- unknown useful features.

Example

A light curve may be characterized by:

- period;
- amplitude envelope;
- phase asymmetry;
- long-term trend;
- outburst morphology.

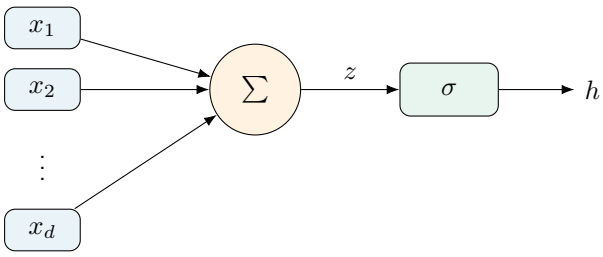
No single raw magnitude point is the full story.

A neuron is an affine map plus a nonlinearity

For input vector $\boldsymbol{x} \in \mathbb{R}^d$,

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b \ , \qquad h = \sigma(z) \ .$$

- $\boldsymbol{w}$: weights;
- b: bias;
- σ: activation function;
- h: transformed feature.



Why activations matter

If we stack only linear transformations,

$$\boldsymbol{h}_1 = \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 \ , \qquad \boldsymbol{h}_2 = \boldsymbol{W}_2 \boldsymbol{h}_1 + \boldsymbol{b}_2 \ ,$$

then

$$\boldsymbol{h}_2 = (\boldsymbol{W}_2 \boldsymbol{W}_1)\boldsymbol{x} + (\boldsymbol{W}_2 \boldsymbol{b}_1 + \boldsymbol{b}_2) \ ,$$

which is still just a linear transformation.

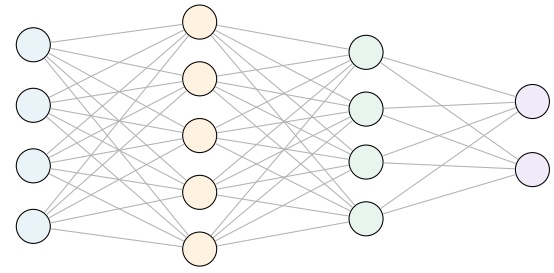
Key point

Depth only becomes useful when nonlinear activations are inserted between layers.

Common choices:

$$\mathrm{ReLU}(z) = \max(0, z) \ , \qquad \tanh(z) \ , \qquad \sigma(z) = \frac{1}{1 + e^{-z}} \ .$$

A multilayer perceptron



$$\boldsymbol{h}^{(\ell+1)} = \sigma\left(\boldsymbol{W}^{(\ell)}\boldsymbol{h}^{(\ell)} + \boldsymbol{b}^{(\ell)}\right) , \qquad \boldsymbol{h}^{(0)} = \boldsymbol{x} .$$

Forward pass for classification

For a K-class astrophysical classifier:

$$\boldsymbol{x} \to \boldsymbol{h}_1 \to \boldsymbol{h}_2 \to \boldsymbol{s} \to \boldsymbol{p} \ .$$

Scores / logits

$$\boldsymbol{s} = \boldsymbol{W}_o \boldsymbol{h}_L + \boldsymbol{b}_o \ .$$

Scores are unconstrained real numbers.

Probabilities

$$p_k = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}} \ .$$

Softmax outputs positive numbers summing to one.

Example

A light curve classifier may output probabilities for RR Lyrae, Cepheid, eclipsing binary, Be star, and non-variable.

Loss functions tell the model what matters

For supervised learning, we minimize a loss:

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}[f_\theta(\boldsymbol{x}_i), y_i] \ .$$

Regression

$$\mathcal{L} = (y - \hat{y})^2 \ .$$

Good for continuous targets such as redshift, stellar mass, or effective temperature.

Classification

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log p_k \ .$$

Cross-entropy penalizes assigning low probability to the true class.

Class imbalance changes the effective objective

If rare objects are scientifically important, ordinary cross-entropy may underweight them.

Weighted binary cross-entropy

$$\mathcal{L}_{\mathrm{WBCE}} = -\alpha y \log p - \beta(1-y)\log(1-p)\ , \qquad \alpha > \beta\ .$$

Useful for:

- rare transients;
- Be-star candidate retrieval;
- exoplanet transit candidates;
- strong-lens searches.

But:

- weights do not fix label noise;
- weights do not fix leakage;
- weights do not fix domain shift;
- threshold still matters.

Backpropagation: the chain rule at scale

Consider a scalar loss $\mathcal{L}$ and a network

$$\boldsymbol{x} \to z_1 \to h_1 \to z_2 \to \hat{y} \to \mathcal{L} \ .$$

The gradient with respect to a parameter w is found by chaining local derivatives:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w} \ .$$

Interpretation

Backpropagation is not a new physical principle. It is efficient bookkeeping for derivatives in a computational graph.

Automatic differentiation

Modern frameworks such as PyTorch and TensorFlow build computational graphs and compute gradients automatically.

What the user writes

- model definition;
- forward pass;
- loss computation;
- optimizer step.

What autograd provides

- gradients of all trainable parameters;
- efficient reverse-mode differentiation;
- GPU-compatible tensor operations;
- modular composition of layers.

`loss.backward()` means: compute $\nabla_\theta \mathcal{L}$.

Gradient descent and neural network training

At iteration t:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \ ,$$

where η is the learning rate.

In practice:

- use mini-batches;
- shuffle training data;
- evaluate validation loss;
- stop before overfitting;
- save model checkpoints.

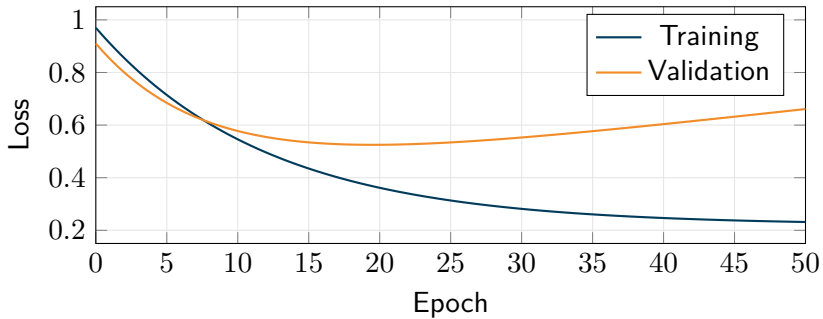
Common optimizers:

- SGD;
- SGD with momentum;
- Adam;
- AdamW.

Astrophysical reminder

The optimizer minimizes a numerical objective, not necessarily the scientific risk you care about.

Training curves: what should students look for?



Typical warning sign

Training loss continues to decrease while validation loss stops improving or increases.

Why neural networks overfit easily

Neural networks often have many parameters and flexible nonlinear transformations.

Overfitting is encouraged by:

- small datasets;
- noisy or ambiguous labels;
- excessive architecture size;
- weak regularization;
- training for too long;
- leakage from preprocessing or splits.

Astronomy-specific risks

- same physical object in train and test;
- same sky field or cluster encoded implicitly;
- cadence tells the model the survey;
- brightness selection mimics class label;
- simulated training set unlike real data.

Regularization toolbox

Method	What it does
Weight decay	Penalizes large weights; often improves smoothness and generalization.
Dropout	Randomly removes hidden units during training; discourages co-adaptation.
Early stopping	Stops training when validation performance no longer improves.
Data augmentation	Expands training examples using transformations that preserve labels.
Batch/layer normalization	Stabilizes hidden activations and optimization.
Smaller architecture	Reduces model capacity when data are limited.

Physical validity matters

Augmentation is only valid if it preserves the astrophysical label. A random image rotation may be valid for galaxy morphology, but a random time reversal may not be valid for an outburst light curve.

Normalization: a numerical and scientific choice

Feature normalization

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} ,$$

helps optimization and prevents large-scale features from dominating.

Leakage warning

Compute μ and σ from training data only.

Astrophysical consequence Normalization can remove or preserve physical information:

- absolute brightness;
- color scale;
- amplitude;
- continuum shape;
- time baseline.

Question

Should a stellar spectrum be continuum-normalized if the continuum contains temperature information?

Representation learning

A neural network does not only map input to output. It learns intermediate representations.

$$\boldsymbol{x} \xrightarrow{E_1} \boldsymbol{h}_1 \xrightarrow{E_2} \boldsymbol{h}_2 \xrightarrow{E_3} \boldsymbol{h}_3 \xrightarrow{C} \hat{y} \ .$$

Encoder

$$\boldsymbol{h} = E_\theta(\boldsymbol{x}) \ .$$

Transforms raw data into a learned feature vector.

Classifier / regressor head

$$\hat{y} = C_\phi(\boldsymbol{h}) \ .$$

Uses the representation to predict the target.

In many modern models, the representation $\boldsymbol{h}$ is as important as the prediction $\hat{y}$.

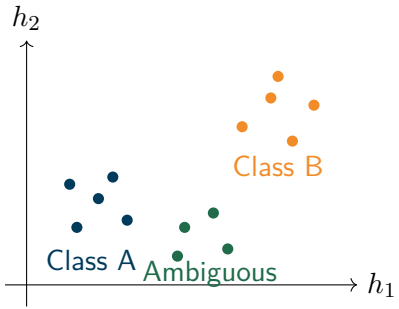
Embeddings as learned coordinates

An embedding is a learned vector representation:

$$\boldsymbol{x}_i \longmapsto \boldsymbol{h}_i \in \mathbb{R}^m \ .$$

Useful embeddings should:

- place similar objects nearby;
- separate relevant physical regimes;
- suppress irrelevant nuisance variation;
- support transfer to related tasks.



What should an embedding ignore?

Relevant variation

Variation connected to the scientific target:

- pulsation morphology;
- disk growth signatures;
- spectral lines;
- galaxy structure;
- transit shape.

Nuisance variation

Variation that can create shortcuts:

- survey identity;
- cadence pattern;
- instrument response;
- magnitude limit;
- preprocessing pipeline;
- sky position or cluster membership.

A good representation is not simply information-rich; it is useful for the intended scientific question.

Representation learning in astrophysical examples

Data	**Possible learned representation**	**Scientific use**
Galaxy image	morphology, asymmetry, bars, spiral structure	galaxy classification, merger search
Spectrum	line patterns, continuum shape, abundance-sensitive regions	stellar parameters, redshift, chemical tags
Light curve	periodicity, bursts, trends, phase morphology	variable-star class, exoplanet candidates
Polarimetry	geometry in Stokes space, variability in $q - u$ plane	disk inclination, scattering geometry
Simulation fields	shocks, turbulence, filaments, vortices	model comparison, surrogate inference

Transfer learning: reuse the representation

Pretraining

Learn an encoder on a large dataset:

$$\boldsymbol{h} = E_{\theta_0}(\boldsymbol{x}) \ .$$

Fine-tuning

Adapt the encoder or head to a smaller astrophysical task:

$$\hat{y} = C_{\phi}(E_{\theta}(\boldsymbol{x})) \ .$$

Useful when:

- labels are scarce;
- raw data are abundant;
- related surveys exist;
- low-level patterns transfer.

Risk

The pretrained representation may transfer survey biases as well as useful structure.

Self-supervised learning in one slide

Self-supervised learning creates a training signal from the data themselves.

Masked modeling

Hide part of the input and ask the model to reconstruct it.

$$\boldsymbol{x}_{\text{masked}} \rightarrow \hat{\boldsymbol{x}}_{\text{missing}} \ .$$

Contrastive learning

Bring related views together and push unrelated views apart.

$$\text{sim}(\boldsymbol{h}_i, \boldsymbol{h}_i^+) > \text{sim}(\boldsymbol{h}_i, \boldsymbol{h}_j^-) \ .$$

Astrophysical examples:

- mask wavelength regions in spectra;
- predict missing light-curve segments;
- match multi-band images of the same object;
- learn embeddings from unlabeled survey data.

Mini-discussion: what did the network learn?

Scenario

A neural network classifies Be-star candidates from light curves with high accuracy on a mixed test split. Later, performance drops when training on one cluster and testing on another.

Discuss in pairs:

1. What shortcuts might the network have learned?
2. Which diagnostic experiment would you run first?
3. Which representation would you inspect?
4. Which metric would convince you the model is scientifically useful?

Architecture encodes assumptions about data geometry

Architecture	Assumption	Astrophysical data
MLP	input is a feature vector; no explicit locality	catalogs, engineered features
CNN	local patterns repeat across position or time	images, spectra, regular light curves
RNN/GRU/LSTM	ordered sequence with memory	time series, token sequences
TCN	temporal locality with long-range receptive field	long light curves, spectra
Transformer	pairwise interactions and flexible context	spectra, irregular time series, multimodal data
Graph neural network	relationships between objects matter	surveys, simulations, catalogs with neighborhood structure

Core idea

Model choice is a scientific modeling decision, not just a software choice.

Convolution: local pattern detection

For a one-dimensional signal $x[t]$ and kernel $k[r]$,

$$(y = k * x)[t] = \sum_{r=-R}^{R} k[r]x[t-r] \ .$$

A convolutional filter can detect:

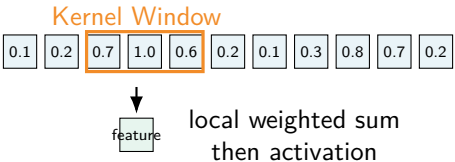
- an absorption line;
- an emission line;
- an edge in an image;
- a transit dip;
- a local burst.

Why shared weights?

The same pattern may occur in multiple positions:

- galaxy feature at different image locations;
- spectral line shifted by redshift;
- local time-series pattern at different epochs.

Convolution as a sliding filter



The same kernel is applied repeatedly, producing a feature map.

Equivariance: why CNNs fit images

A transformation T applied to the input should produce a corresponding transformation of the features:

$$E(T\boldsymbol{x}) \approx TE(\boldsymbol{x}) \ .$$

Translation equivariance

If a spiral arm shifts slightly in the image, the feature map shifts too.

Not all symmetries are automatic

Ordinary CNNs are not automatically rotation-, scale-, or redshift-invariant.

Astrophysical question: Which transformations preserve the label?

- galaxy rotation on the sky may preserve morphology;
- spectral redshift does not preserve observed wavelength location;
- time reversal may not preserve outburst physics.

CNN building blocks

Convolutional layer

Learns local filters.

Activation

Introduces nonlinearity.

Pooling or stride

Reduces resolution and increases effective receptive field.

Normalization

Stabilizes training.

Residual connection

Allows information and gradients to pass more easily:

$$\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_{\ell} + F(\boldsymbol{h}_{\ell}) \ .$$

CNNs for astronomical images

Good use cases:

- galaxy morphology;
- strong-lens detection;
- supernova host images;
- solar images;
- simulation snapshots;
- astronomical cutouts.

Key checks

- Is the train/test split object-level?
- Are augmentations physically valid?
- Is performance stable across magnitude, redshift, S/N?
- Are artifacts or masks predictive?
- Does the model generalize across surveys?

1D CNNs for spectra

A spectrum is a one-dimensional signal over wavelength:

$$\lambda \longmapsto F(\lambda) \ .$$

Local filters can learn:

- line depths;
- line widths;
- blends;
- continuum changes;
- local correlations.

Preprocessing choices

- continuum normalization;
- rest-frame correction;
- wavelength grid;
- masking bad pixels;
- resolution matching.

These choices can dominate the result.

1D CNNs for light curves: useful but not automatic

A regular 1D CNN assumes samples are on a regular grid:

$$(t_1, t_2, \ldots, t_T) \ , \qquad t_{j+1} - t_j = \Delta t \ .$$

Astronomical light curves are often:

- irregularly sampled;
- gapped;
- multi-band;
- heteroscedastic;
- variable length.

Common solutions

- resample or interpolate;
- phase-fold periodic signals;
- use Δt as an input channel;
- use masks;
- use features plus sequence model;
- use attention with time encodings.

Sequence learning problem

Many astronomical datasets are ordered sequences:

$$\{(t_j, x_j, \sigma_j)\}_{j=1}^{T} \ .$$

Examples:

- light curves;
- spectra as wavelength sequences;
- polarimetric time series;
- alert streams;
- simulation trajectories.

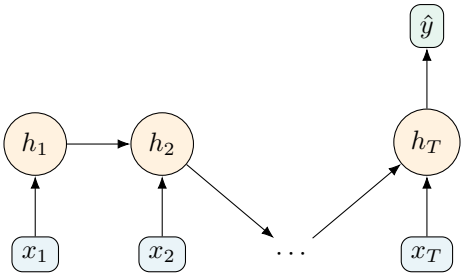
Important questions:

- Are time intervals regular?
- Does order matter?
- Are long-range dependencies important?
- Is the target local or global?
- Are missing observations informative?

Recurrent neural networks

An RNN updates a hidden state as it reads a sequence:

$$\boldsymbol{h}_t = \sigma(\boldsymbol{W}_x \boldsymbol{x}_t + \boldsymbol{W}_h \boldsymbol{h}_{t-1} + \boldsymbol{b}) \ , \qquad \hat{y} = C(\boldsymbol{h}_T) \ .$$



GRUs and LSTMs: gated memory

Plain RNNs can struggle with long-range dependencies.

Idea of gating

Let the model decide what to keep, forget, and update.

Useful for:

- long light curves;
- recurrent variability;
- evolving disk states;
- transient precursors;
- time-ordered polarimetry.

Limitations:

- sequential computation can be slow;
- long sequences remain difficult;
- irregular cadence needs care;
- hidden states can be hard to interpret.

Temporal convolutional networks

A temporal convolutional network (TCN) uses stacked 1D convolutions, often with dilation.

Dilated convolution

A filter can skip inputs to increase its receptive field:

$$y[t] = \sum_{r=0}^{R} k[r]x[t - dr] \ ,$$

where d is the dilation.

Advantages:

- parallelizable;
- stable training;
- captures local and wider temporal patterns;
- often strong baseline for sequences.

Astrophysical uses:

- phase-folded light curves;
- spectra;
- binned time series;
- synthetic simulation trajectories.

Attention: compare every element to every other element

Self-attention transforms a sequence by allowing each element to attend to other elements.

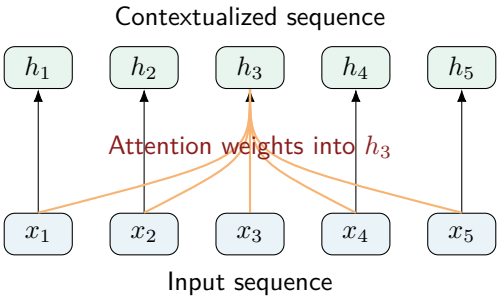
$$Q = XW_Q \ , \qquad K = XW_K \ , \qquad V = XW_V \ .$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \ .$$

Interpretation

Each output vector is a weighted mixture of value vectors. The weights are determined by query-key similarity.

Attention diagram



The representation of one element can depend directly on all other elements.

Why transformers need positional information

Self-attention alone is permutation-invariant over sequence elements. It does not know order unless order is encoded.

Position encoding

Add or concatenate information about position:

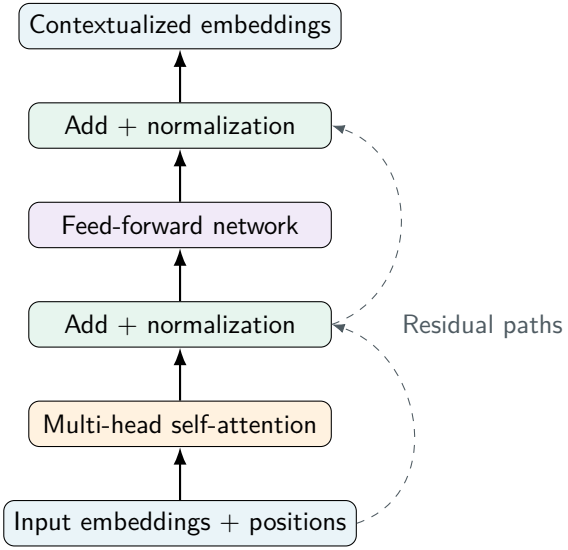
$$\tilde{\boldsymbol{x}}_t = \boldsymbol{x}_t + \boldsymbol{p}_t \ .$$

Astrophysical variants

- wavelength position;
- observation time;
- phase;
- time gap Δt;
- band/filter identity;
- sky or survey metadata.

For irregular light curves, time encoding is not optional.

Transformer encoder block



Multi-head attention

Instead of one attention operation, use several heads:

$$\text{head}_m = \text{Attention}(QW_m^Q, KW_m^K, VW_m^V) \ .$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_M)W^O \ .$$

Possible intuition:

- one head tracks local line shape;
- another tracks long-range continuum;
- another tracks band-to-band relation;
- another tracks outburst context.

Caution

Attention weights are not automatically explanations of physical causality.

Transformers in astrophysics: when are they attractive?

Strengths:

- flexible long-range interactions;
- works with variable-length inputs;
- handles multimodal tokens;
- supports pretraining;
- useful for large unlabeled datasets.

Weaknesses:

- data-hungry;
- computationally expensive;
- can overfit small datasets;
- positional/time encoding is delicate;
- interpretability is not automatic.

Rule of thumb

Use a transformer when flexible context or pretraining is genuinely useful, not because it is fashionable.

Multimodal astrophysical learning

Many astrophysical objects are observed through multiple views.

$$\hat{y} = F\left[E_{\text{image}}(\boldsymbol{x}_{\text{image}}), E_{\text{spec}}(\boldsymbol{x}_{\text{spec}}), E_{\text{lc}}(\boldsymbol{x}_{\text{lc}}), E_{\text{meta}}(\boldsymbol{x}_{\text{meta}})\right] \ .$$

Examples:

- image + spectrum;
- light curve + colors;
- photometry + polarimetry;
- simulation field + physical parameters;
- alert packet + host-galaxy context.

Key danger

A model may exploit the easiest modality, even if it is not the most physical one.

Choosing an architecture for each data type

Problem	Reasonable starting model	Stronger model
Catalog classification	logistic regression, random forest	MLP, gradient boosting
Galaxy morphology	small CNN	pretrained CNN / vision transformer
Stellar spectra	PCA + regressor, 1D CNN	transformer / pretrained spectral encoder
Variable light curves	features + RF, 1D CNN on phase-folded data	GRU/TCN/transformer with time encodings
Exoplanet transits	feature model, 1D CNN	local/global two-branch model
Polarimetry	linear model in (q, u)	MLP / multimodal model with photometry
Anomaly detection	PCA + isolation forest	autoencoder / self-supervised embedding

Start simple. Then add complexity only if it answers a real limitation.

A model-selection checklist

Before choosing a deep architecture, ask:

1. What is the unit of independence: object, exposure, spectrum, event, pixel?
2. What is the natural data geometry: vector, image, sequence, graph, field, multimodal set?
3. What transformations should preserve the label?
4. Which nuisance variables could become shortcuts?
5. How large is the labeled dataset?
6. Is pretraining possible or useful?
7. What baseline must the model beat?
8. What failure mode would make the model scientifically unusable?

Deep learning diagnostics beyond accuracy

Numerical diagnostics

- training/validation curves;
- confusion matrices;
- precision-recall curves;
- calibration plots;
- residual plots;
- ablation studies.

Astrophysical diagnostics

- examples of failures;
- performance versus S/N;
- performance versus magnitude/redshift;
- per-source/per-survey metrics;
- physically meaningful feature sensitivity;
- expert review of edge cases.

Interpretability methods: useful, but limited

Method	What it can show	Caution
Permutation importance	which features matter for a trained model	correlated features can confuse importance
SHAP-style methods	contribution of input features to a prediction	not a causal proof
Saliency maps	pixels or wavelengths that affect output	can be noisy and unstable
Attention weights	which tokens influence representations	attention is not necessarily explanation
Ablation	whether removing a region hurts performance	depends on ablation design
Embedding plots	clusters and separations in representation space	projection can distort geometry

Interpretability should be combined with astrophysical sanity checks.

Failure modes in deep astrophysical models

Data failure

- leakage;
- label noise;
- sample selection bias;
- missing uncertainty model;
- unrepresentative simulations.

Model failure

- overfitting;
- shortcut learning;
- poor calibration;
- out-of-distribution collapse;
- unstable interpretation.

A deep model can fail scientifically even when the code runs perfectly.

Micro-exercise: match data, architecture, and risk

For each case, choose a starting model and name one failure mode.

Case	Starting architecture	Possible failure mode
Galaxy cutouts from two surveys		
Irregular Be-star light curves		
Broadband photometric redshifts		
Stellar spectra with bad pixels		
Polarimetric $q - u$ trajectories		
Rare exoplanet transit candidates		

Discuss: Which cases require deep learning, and which only require better representation and validation?

Bridge to the hands-on notebooks

In Week 2, you will implement these ideas in notebooks:

Monday lab

- tabular catalog ML;
- classical baselines;
- PyTorch MLP;
- metrics and thresholds.

Wednesday lab

- spectra and time series;
- engineered features vs 1D CNNs;
- representation comparison;
- error analysis.

Friday lab

Polarimetry, mini-project design, and final scientific interpretation.

Recommended exercises after Wednesday

Use Exercise Set 1, Wednesday section:

- derive the forward pass of an MLP;
- explain why stacked linear layers collapse to one linear layer;
- interpret cross-entropy for a variable-star classifier;
- compare CNNs, RNNs, TCNs, and transformers for light curves;
- design a physically valid augmentation strategy;
- identify possible shortcut variables in an astrophysical deep-learning pipeline.

Preparation for Friday

Read about uncertainty, calibration, domain shift, and generative AI before the final theory day.

Closing message

Deep learning is not magic.

It is a way to learn useful representations by optimizing a chosen objective under architectural assumptions.

For astrophysics

The central question is not only:

Does the model predict well?

It is also:

Did the model learn structure that is scientifically meaningful?

One-slide glossary

Term	Meaning
Activation	Nonlinear function applied after an affine transformation.
Backpropagation	Efficient computation of gradients by the chain rule.
Embedding	Learned vector representation of an input.
Encoder	Network component that maps data into a representation.
CNN	Neural network using learned local convolutional filters.
RNN	Sequence model with a hidden state updated over time.
TCN	Sequence model using temporal convolutions, often dilated.
Attention	Mechanism that forms weighted mixtures based on pairwise similarity.
Transformer	Architecture built from self-attention, feed-forward layers, normalization, and residual connections.
Shortcut learning	Using nuisance correlations rather than the intended physical signal.

Optional reading and preparation

- Review the Week 1 exercise set, especially the neural-network and architecture questions.
- Before the Week 2 notebooks, make sure you understand:
  - train/validation/test split;
  - PyTorch tensors;
  - loss, optimizer, and training loop;
  - confusion matrix and precision-recall curve;
  - why preprocessing must be fitted on training data only.
- For mini-project preparation, choose one data geometry: catalog, image, spectrum, light curve, polarimetry, or anomaly detection.